Sprint Review and Retrospective: SNHU Travel

For ChadaTech's SNHU Travel project, I practiced Scrum-Agile development of a travel application. Throughout CS 250, I played different roles and created project artifacts rather than joining a live team. This review explains what I learned and how I would apply those lessons as Scrum Master.

Applying Scrum-Agile Roles

As a Product Owner, I prioritized five user stories in the Product Backlog: viewing the top five destinations, saving preferences, finding deals, choosing a vacation type, and setting a price limit. Starting with the destination list, the first feature is manageable.

As a Developer, I used Java Swing to create a basic destination slideshow with information about Tokyo, Paris, Rome, Madrid, and Honolulu. The Tester role was just as important because it checked whether the application worked as intended. During testing, I saw that the slideshow displayed five destinations and that the Next and Previous controls worked, but some pictures were missing. This showed me that having a working feature does not automatically mean every acceptance criterion is met. The Scrum Master supports the team by clarifying priorities and removing blockers. Testing is part of the Developers' work rather than a separate Scrum accountability.

Completing User Stories

User stories helped turn the general goal of "make travel easier" into work that could be planned, built, and checked. For the top five destinations story, users needed to see a destination name, a

short description, and an image as they moved through the list. The original requirements also included a travel-package link, which still needed verification. Specific behaviors made the work testable.

At Sprint Planning, the team would select the story and define completion. Building and testing in short cycles exposed missing pictures early, allowing the defect to be corrected and retested. I also wrote test cases for saving preferences and viewing relevant deals, but those were planning and testing artifacts, not proof that those extra features were implemented. A story is complete only when its acceptance criteria and Definition of Done are met.

Handling Interruptions and Changes

The biggest interruption came when SNHU Travel changed the focus from general vacations to detox and wellness vacations while keeping the original timeline. Waterfall could require revising approved requirements. Scrum allows the Product Owner to clarify the new goal and reorder the backlog without restarting the whole application.

The existing slideshow design was useful because its basic navigation could stay the same while destination images, descriptions, and offers were revised for the new audience. In my developer reflection, I focused on getting updated user stories and acceptance criteria from the Product Owner and new test cases from the Tester before continuing. I would also raise the schedule risk early and move lower-value stories out of scope instead of silently adding new work. The change still meant rework and choices about what to defer.

Communication and Collaboration

The course used simulated role-based work, so the following are examples based on the communication I practiced in my journal rather than transcripts from a live team.

One message I would send to the Product Owner is: "Since the client wants wellness to travel now, which destination details are required for the next release, and which older stories should move down the backlog?" This question is effective because it asks for a decision about scope instead of leaving the Developer to guess.

A second message to the Tester would be: "The Next and Previous buttons work, but some destination pictures did not appear. Can we retest each slide after the image paths are corrected and confirm the updated wellness descriptions?" This identifies the defect, proposes the next step, and invites a shared check of acceptance criteria.

A Daily Scrum surfaces progress and blockers. A Sprint Review would gather feedback on the slideshow; a Retrospective would ask how to spot broken images earlier.

Organizational Tools and Scrum Events

The Product Backlog was the most useful organizational tool because it kept the user's stories and business priorities together. The Sprint Backlog would make the selected tasks and fixes visible during the sprint, while test cases served as a checklist for expected behavior. A "To Do," "In Progress," "Testing," and "Done" board would make progress visible in a live team; I did not use a commercial board in this simulation.

Scrum events connect the tools: Sprint Planning sets the goal; the Daily Scrum tracks progress; the Sprint Review checks the increment with stakeholders; and the Retrospective improves the process.

For this project, I would add an image-loading check to each test cycle and ask the Product Owner to confirm changed acceptance criteria before work resumes. This is a practical improvement that came directly from the problems I encountered.

Evaluating the Agile Process

Scrum-Agile worked well for SNHU Travel because the product was still taking shape and customer priorities changed. Its biggest advantages were shorter feedback cycles, visible priorities, and flexibility. The slideshow could be demonstrated early, the missing pictures could be discovered before a broader release, and the wellness pivot could be handled through backlog refinement. The approach also encouraged me to think about the application from several perspectives instead of focusing only on the code.

There were disadvantages too. Agile needs regular communication and clear acceptance criteria; otherwise, changes can create confusion or scope of creep. Reprioritizing work does not make it free, and repeated adjustments can make delivery estimates less certain. The class simulation did not reproduce all the coordination challenges of a live team.

Even with those limits, I believe Scrum-Agile was the better choice for this project. SNHU Travel was exploring a new customer-facing product, and the shift to wellness travel showed why frequent review and adaptation were more useful than following one fixed plan.

My main takeaway is that an effective Scrum team combines flexibility with honest testing, clear priorities, and regular communication.